

Name: Manaswi Kabadi

Class: D20C

Roll No.: 15

Blockchain Experiment 6

Aim:

Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a Voting Smart Contract, require can be used to check:

- Whether the person calling the function has the right to vote
(require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, require statements enforce security, correctness, and reliability in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier.

2. Keywords: mapping, storage, and memory

- mapping:

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like Ballot, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them gas efficient for lookups but limited for enumeration.

- storage:

In Solidity, storage refers to the permanent memory of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is

more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory:**

In contrast, memory is temporary storage, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used. Thus, a smart contract developer must balance between storage and memory to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string.

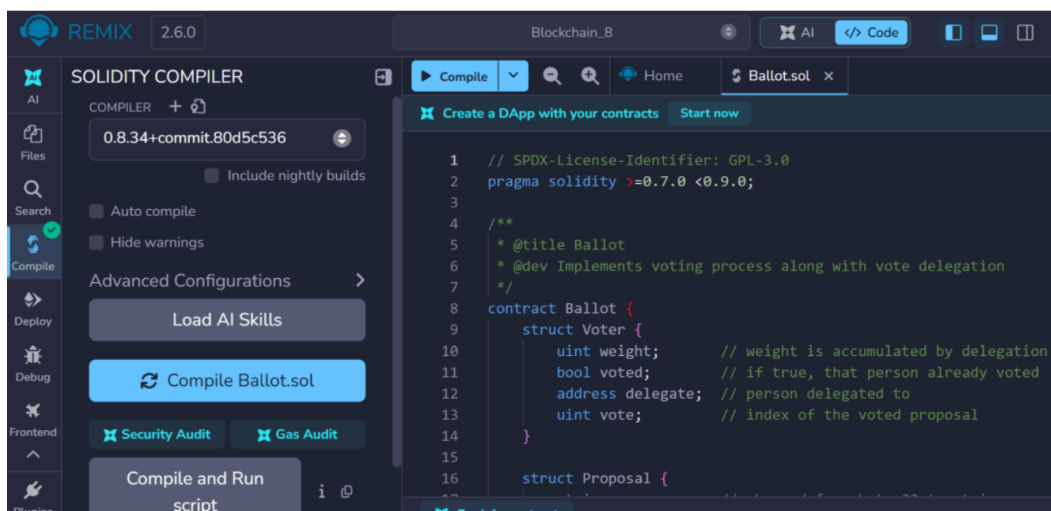
The reason lies in efficiency and gas optimization.

- bytes32 is a fixed-size type, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- string is a dynamically sized type, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the Web3 Type Converter help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

Output:



REMIX 2.6.0 Blockchain_8 AI </> Code

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Remix VM Osaka

Account 1 99.999 ETH
0x5B3...eddC4

Deploy Deployed contracts 1 Transactions h

Deployed Contracts 1 + Add Contract

Interact with a deployed contract

Ballot Remix VM

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
6  * @dev Implements voting process along with vote delegation
7  */
8 contract Ballot {
9     struct Voter {
10         uint weight; // weight is accumulated by delegation
11         bool voted; // if true, that person already voted
12         address delegate; // person delegated to
13         uint vote; // index of the voted proposal
14     }
15
16     struct Proposal {
```

REMIX 2.6.0 Blockchain_8 AI </> Code Sign In

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Remix VM Osaka

Account 1 99.999 ETH
0x5B3...eddC4

Deploy Deployed contracts 1 Transactions h

Deployed Contracts 1 + Add Contract

Interact with a deployed contract

Ballot Remix VM

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
6  * @dev Implements voting process along with vote delegation
7  */
8 contract Ballot {
9     struct Voter {
10         uint weight; // weight is accumulated by delegation
11         bool voted; // if true, that person already voted
12         address delegate; // person delegated to
13         uint vote; // index of the voted proposal
14     }
15
16     struct Proposal {
```

Explain contract AI copilot

Listen on all transactions Filter with transaction hash or address

interface

Right-click on a JavaScript file in the file explorer and then click 'Run'

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0x605...e7b4b Debug

CALL [call] from: 0x5B380a6a701c568545dcfcB03fC875f56beddC4 to: Ballot.proposals(uint256) data: 0x013...00001 Debug

REMIX 2.6.0 Blockchain_8 AI </> Code Sign In

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Remix VM Osaka

Account 1 99.999 ETH
0x5B3...eddC4

Deploy Deployed contracts 1 Transactions h

Deployed Contracts 1 + Add Contract

Interact with a deployed contract

Ballot Remix VM

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
6  * @dev Implements voting process along with vote delegation
7  */
8 contract Ballot {
9     struct Voter {
10         uint weight; // weight is accumulated by delegation
11         bool voted; // if true, that person already voted
12         address delegate; // person delegated to
13         uint vote; // index of the voted proposal
14     }
15
16     struct Proposal {
```

Explain contract AI copilot

Listen on all transactions Filter with transaction hash or address

• ethers.js

Type the library name to see available commands.

[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0x605...e7b4b Debug

CALL [call] from: 0x5B380a6a701c568545dcfcB03fC875f56beddC4 to: Ballot.proposals(uint256) data: 0x013...00001 Debug

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xed2...8a76d value: 0 wei data: 0x9e7...35cb2 logs: 0 hash: 0x5d4...a37ff Debug

REMIX 2.6.0 Blockchain_B AI AI Code Sign In

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Remix VM Osaka

Account 2 99.999 ETH
0xAb8...35cb2

Deploy Deployed contracts Transactions

vote (uint256)

1 Transact

Calldata Params

chairperson Call

proposals (uint256) Call

1

Calldata Params

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
6  * @dev Implements voting process along with vote delegation
7  */
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: Ballot.constructor value: 0 wei data: 0x68...0000 logs: 0 hash: 0x605...e7b4b Debug

CALL [call] from: 0x5B380a6a701c568545dCf803Fc875F56beddC4 to: Ballot.proposals(uint256) data: 0x013...00001 Debug

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xed2...8a76d value: 0 wei data: 0x9e7...35cb2 logs: 0 hash: 0x5d4...a37ff Debug

[vm] from: 0xAb8...35cb2 to: Ballot.vote(uint256) 0xed2...8a76d value: 0 wei data: 0x012...00001 logs: 0 hash: 0x5f0...f23bb Debug

REMIX 2.6.0 Blockchain_B AI AI Code Sign In

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Remix VM Osaka

Account 2 99.999 ETH
0xAb8...35cb2

Deploy Deployed contracts Transactions

Calldata Params

voters (address) Call

0xAb8483F64d9C6d1EcF9b849Ae677D03315835cb2

Calldata Params

uint256: weight 1

1: bool: voted true

2: address: delegate 0x00

3: uint256: vote 1

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
6  * @dev Implements voting process along with vote delegation
7  */
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

CALL [call] from: 0x5B380a6a701c568545dCf803Fc875F56beddC4 to: Ballot.proposals(uint256) data: 0x013...00001 Debug

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xed2...8a76d value: 0 wei data: 0x9e7...35cb2 logs: 0 hash: 0x5d4...a37ff Debug

[vm] from: 0xAb8...35cb2 to: Ballot.vote(uint256) 0xed2...8a76d value: 0 wei data: 0x012...00001 logs: 0 hash: 0x5f0...f23bb Debug

CALL [call] from: 0xAb8483F64d9C6d1EcF9b849Ae677D03315835cb2 to: Ballot.voters(address) data: 0xa3e...35cb2 Debug

REMIX 2.6.0 Blockchain_B AI AI Code Sign In

DEPLOY & RUN TRANSACTIONS

Environment Fork Reset

Remix VM Osaka

Account 1 99.999 ETH
0x5B3...eddC4

Deploy Deployed contracts Transactions

giveRightToVote (address) Transact

0x4B20993Bc481177ec7E

Calldata Params

vote (uint256) Transact

1

Calldata Params

chairperson Call

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title Ballot
6  * @dev Implements voting process along with vote delegation
7  */
```

Explain contract AI copilot

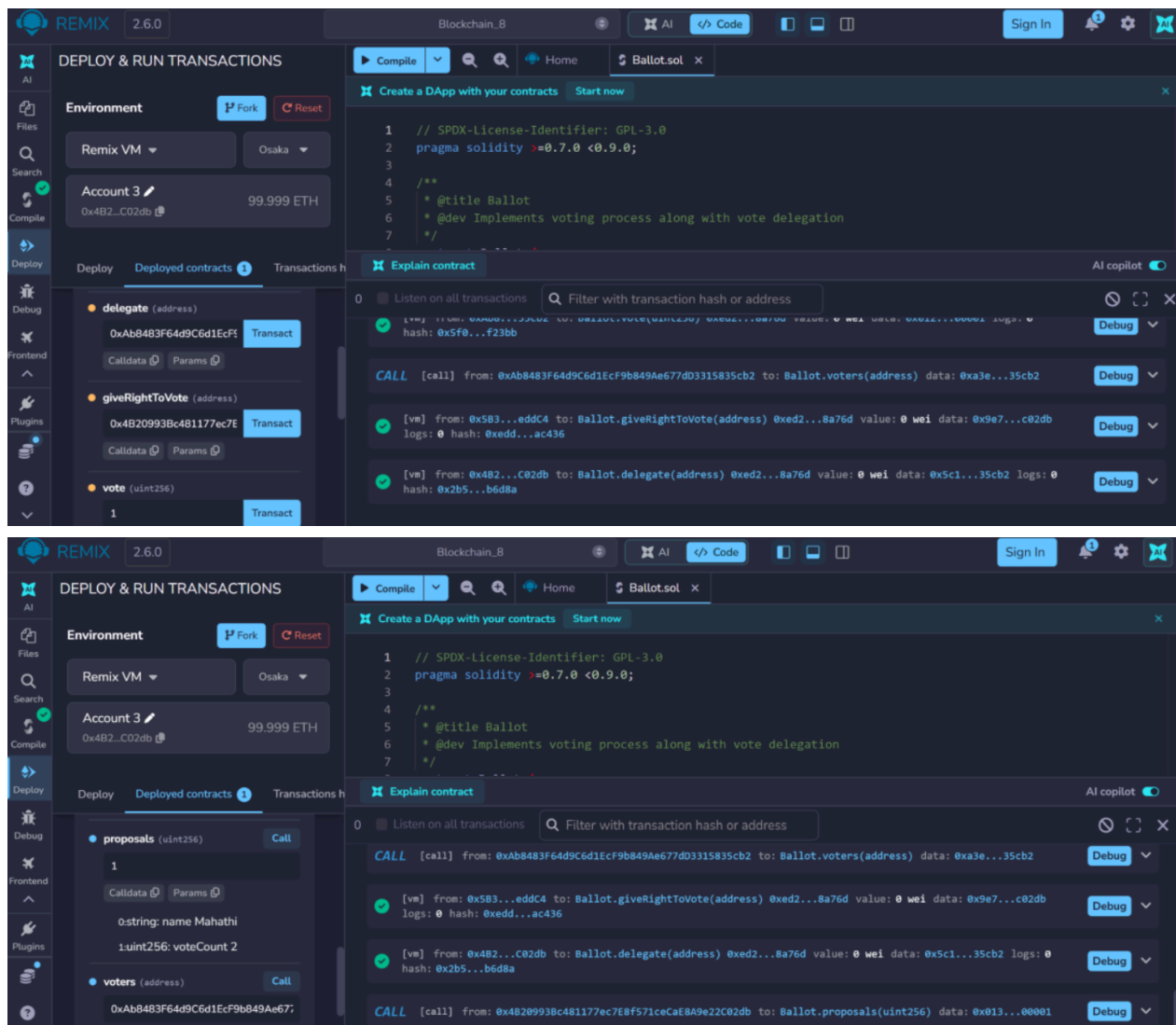
0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xed2...8a76d value: 0 wei data: 0x9e7...35cb2 logs: 0 hash: 0x5d4...a37ff Debug

[vm] from: 0xAb8...35cb2 to: Ballot.vote(uint256) 0xed2...8a76d value: 0 wei data: 0x012...00001 logs: 0 hash: 0x5f0...f23bb Debug

CALL [call] from: 0xAb8483F64d9C6d1EcF9b849Ae677D03315835cb2 to: Ballot.voters(address) data: 0xa3e...35cb2 Debug

[vm] from: 0x5B3...eddC4 to: Ballot.giveRightToVote(address) 0xed2...8a76d value: 0 wei data: 0x9e7...c02db logs: 0 hash: 0xedd...ac436 Debug



Conclusion:

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.